C-MCPN: The Clingo McCulloch-Pitts Network Editor

An Answer-Set Programming Framework for Artificial Neural Networks

David Gonzalez[1], Joshua T. Guerin, Ph.D.[2]
Mathematical Sciences
University of Northern Colorado
Greeley, CO 80639
[1]********@bears.unco.edu
[2]********@unco.edu

Abstract

Artificial Intelligence, in particular generative AI (gen-AI) and Large Language Models (LLMs), have engaged the public in ways that mirror other critical advances in computing technologies like the PC and smartphone revolutions of the 1980s and 2000s, and the rapid expansion of the Internet over the past three decades. Entire industries, including education, are attempting to simultaneously attempting to predict and shape the future directions of these and related technologies.

While Machine Learning (ML) technologies like LLMs have a profound capacity to integrate AI techniques into modern life, work, and society, the underlying technologies such as Artificial Neural Networks (ANNs) have presented serious issues concerning both short and long-term sustainability. These issues come in a number of forms: footprint for data centers, waste artifacts such as heat and noise, and they require a rather significant footprint regarding precious resources such as energy (infrastructure) and clean water for cooling.

At the same time, advanced declarative languages have made remarkable headway towards generalized AI solutions that are designed with efficiency in mind. In particular, Answer Set Programming (ASP) languages are designed to solve computationally difficult (NP-Complete and NP-Hard) with limited available resources. We hope that by merging technologies from other areas of AI, like ASP languages and solvers, that we may be able to accelerate the learning and inference processes for ANNs, thereby creating a more sustainable computing environment as these technologies continue to explode in their growth.

In this paper we discuss our work creating a representation for ANNs in the ASP dialect of Clingo. As an example domain, we use the widely-studied area of digital logic for hardware construction. We also introduce C-MCPN: The Clingo McCulloch-Pitts Network Editor. C-MCPH is a utility written in Python to assist in our research, and serves as a lightweight, graphical Integrated Development Environment (IDE) for developing these digital-logic based ANNs, including the ability to specify more general neuron structure and activation outside of logical gates.

1 Introduction

Artificial Intelligence systems have had a significant impact on the modern world over the decades. Some of the many advancements across various fields include (however certainly not limited to) conversational agents, marketing, medicine, and insurance (Nössig et al., 2024–need cite). Historically many of these advances and their impact on shaping the modern world have occurred in the background, improving our systems and applications iteratively and often at the periphery of the thinking of the general public. With new advances in AI, in particular machine learning methods (ML) and generative AI (in particular Large Language Models or LLMs), these systems have come to the forefront of public consciousness in a way that is likely to remain for the foreseeable future.

While these AI systems have the potential for significant advancement, a number of contemporary models come with significant drawbacks. In particular, the necessary growth of AI data centers and their associated infrastructure needs have caused significant stress for many local populations [Need citation]. These issues are leading to worsening availability of resources for the general public, and lead directly to the potential of significant environmental and health impacts (Bush 2025–need cite).

For continued advancement to avoid outpacing our environmental needs, significant research is necessary to bring down their respective footprints. Artificial Neural Network (ANN) technologies have made many of these technological advances possible, however drawbacks concerning their efficiency are a continued cause for concern regarding overall sustainability [cite?].

Answer Set Programming (ASP) languages often live at a different end of the research spectrum. These languages are designed to solve NP-Complete and NP-Hard problems in a way that is fast and efficient. In this paper we conduct investigate how to merge these schools of thought by building, representing, and performing inference over ANNs, specifically McCulloch-Pitts Networks (MPNs) [cite?]. To this end we introduce `C-MCPN`: The *Clingo McCulloch-Pitts Network Editor.*

In this paper we will introduce basic techniques for modeling ANNs in the ASP language Clingo, and demonstrate how `C-MCPN` can be used to operate over ANN models.

2 Background

2.1 Artificial Neural Networks (ANNs)

2.1.1 Efficiency and Environmental Immpacts

2.2 Answer Set Programming (ASP)

Symbolic AI has served as a foundation for many of the earliest AI models and advancements. This form of AI relies on manipulation of symbols to represent knowledge and perform inference over models for the purpose of reasoning. These types of AI are explicitly rule-based, meaning that their decision-making systems are much more deterministic (Bhuyanet al., 2024; Liang et al., 2025; Meli et al., 2024–need cite).

The Prolog language is a classical example of these sorts of declarative (logical) programming language [cite]. Its syntax and semantics are largely derived from quantified, predicate logic, and has had a profound impact on the presentation of logic programming that continues into modern systems. Similarly, Boolean Satisfiability (SAT) solvers have served as a lower-level, contemporary form of logical programming. Throughout the 1990s and into the 2000s, these systems have expanded massively in their ability to solve large problems efficiently.

Modern ASP systems combine a simple, yet powerful syntax that is inspired by Prolog, with the efficiency of SAT systems. This empowers the user to write concise programs that can solve computationally-difficult problem instances with rather remarkable speed and efficiency.

2.2.1 Syntax and Semantics

In this section we will elaborate basic features of the Clingo language [cite]–a contemporary ASP language that serves as the basis for `C-MCPN`. In particular, we describe the nature of *facts*, *rules*, and *cardinality constraints* that are used in our representations[1].

The most atomic units in Clingo are *facts*, presented as predicate-based statements. The basic syntax is in the form `property(atom)`, which ascribes a property, `property`, to a symbol `atom`.

E.g.,

[1]A full review of the ASP family's syntax and semantics is outside the scope of this work, we seek to provide minimal instruction. For users with previous experience in Prolog or a similar logical dialect, the syntactic roots of ASP languages should be at somewhat familiar.

```
red(apple).
```

is a complete statement (and program) in Clingo, attributing a property of red to an atom named `apple`. You will note that in this, and future examples, our code operates in a declarative, and non-algorithmic context. Rather than providing discrete instructions to the processor regarding specific steps to solve a problem, these logical languages allow us to *describe* the nature of the world and the problem that we wish to solve, as well as what a solution *looks like* in our domain. It is the role of the *solver* to find a solution that matches our description.

Further knowledge can be *inferred* through combinations of predicates that create *rules* of the form: `consequent :- antecedent`, mirroring the *implication* from formal logic (e.g., $consequent \leftarrow antecedent$). Such rules specify that the *truth* of the `consequent` follows logically from the truth of the `antecedent`.

For example,

```
eat(apple) :- red(apple), ripe(apple).
```

describes a preference over what fruit will be eaten. Specifically, the modeled preference indicates that "if an apple is red and it is also ripe, then we eat the apple."

More generally, we often phrase rules around *variables* for a greater degree of generality in our software. In this way we can describe the nature of entire sets of items, rather than specific atoms. These are identified by an uppercase starting symbol, and are treated as *patterns* to be matched by the system (i.e., variables can match specific atoms based on context).

I.e.,

```
eat(Fruit) :- red(Fruit), ripe(Fruit).
```

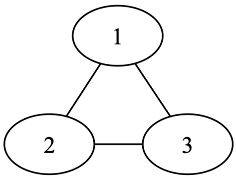
In this case, `Fruit` would match any contextually-relevant symbol, like the atom `apple` that we introduced earlier. In this case the use of variables could extend the notions of edibility, redness, and ripeness to other possible fruits.

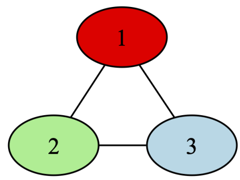
2.2.2 Problem Modeling in Clingo: 3-Colorability

As an illustrative example regarding how to model problems in Clingo, we will use the graph 3-Colorability problem, as it is presented in *ASP∀: Answer Set Programming "Algorithms"* (Guerin et al., 2026–need cite). ASP∀ is an Open Educational Resource (OER) for answer set programming in the Clingo dialect. These implementations provide minimal examples of how to model problems in Clingo, and graph-based implementations are of particular interest to us as ANNs are also graphical in nature.

The 3-Colorability problem is deceptively easy to state, but is computationally expensive to solve exactly. A simplified statement of the problems is "Given a graph, G, assign one of 3 colors to the vertices of G such that no two adjacent vertices share a color."

E.g., Consider the following graph and proposed solution:





Need to fix formatting.

In this particular case, color assignment is rather trivial–any assignment of all three colors to the vertices will lead to a valid coloring.

The implementation for 3-Colorability provided by ASP∀ is specified in terms of an *instance* as well as a *solution* file. The instance encodes input information in the form of predicates[2], and the solution provides a general implementation for inference over any provided instance model.

```
node(1..3).
node(1, 2).
node(1, 3).
node(2, 3).

% Set of available colors.
  color(1..3).

% Edges are symmetric.
% Assumes potentially directed description of an undirected graph.
edge(M, N) :- edge(N, M).

% Each vertex is assigned a single color.
```

[2]Unlike many programming language paradigms, ASP languages often lack traditional IO systems, moving these features to dedicated parsers in other languages.

```
1 { color(N, C) : color(C) } 1 :- node(N).

% No two adjacent vertices can have the same color.
:- edge(N,M), color(N, C), color(M, C).

% Output color information.
#show color/2.
```

```
clingo version 5.6.2

Reading from clingo_test.lp

Solving...

Answer: 1

color(1,2) color(2,3) color(3,1)

SATISFIABLE


Models : 1+
Calls : 1
Time : 0.013s (Solving: 0.00s 1st Model: 0.00s Unsat: 0.00s)
CPU Time : 0.002s
```

3 Methodology

3.1 Constructing the Logical Network

The bulk of our time on this project was spent on constructing the framework for logical networks, and testing it on an 8-bit adder. We chose this example because it is a simple problem that is still complex enough to demonstrate the capabilities of our framework.

A natural starting point for the framework was to create the structure of a sample graph, a 1-bit adder. It is easy to add too much information to each fact, and our final implementaion ended up with a much more minimalistic approach. Clingo is most effective with simple facts that encode a lot of information, marking one of the contributions our framework has to the field of both logic programming and neural networks.

For our example network we used neurons with three types of well established logic gates: AND, OR, and XOR. In Table 1 we show the definitions of these gates, and their truth tables. Using a combination of these and more gates, one is able to construct complex logical operations[3]. Moreover, we believe that the C-MCPN Framework has the potential to be used for a wide range of applications in the future, especially if its capabilities are further explored and developed.

AND			**OR**			**XOR**		
$g(\vec{x}) = x_1 \cdot x_2,\ \theta = 1$			$g(\vec{x}) = x_1 + x_2,\ \theta = 1$			$g(\vec{x}) = \lvert x_1 - x_2 \rvert,\ \theta = 1$		
Activation:						$f(\vec{x}) = \begin{cases} 0 & \text{if } g(\vec{x}) < \theta, \\ 1 & \text{if } g(\vec{x}) \geq \theta. \end{cases}$		
AND			OR			XOR		
x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$	x_1	x_2	$f(\vec{x})$
T	T	T	T	T	T	T	T	F
T	F	F	T	F	T	T	F	T
F	T	F	F	T	T	F	T	T
F	F	F	F	F	F	F	F	F

Table 1: McCulloch-Pitts Neuron Definitions and Truth Tables.

3.2 Constructing the Editor

Alongside the framework, we also built an editor for constructing and visualizing logical networks using the C-MCPN Framework. It is an entirely local desktop application which we felt would be more accessible to a wider range of users. The framework works well in a terminal, but having a live visual representation of the networks is a powerful tool for understanding and debugging them.

The editor is currently built in Python[4] using the DearPyGui library, allowing us to take advantage of its node-based editing capabilities. This meant we could keep the dependencies of the editor to a minimum, and focus on building the features we wanted.

Building the editor in this way also allowed us to easily integrate it with the C-MCPN Framework, as we could directly use the same two-layered data structures for both; one for the network architecture, and one for the values of each neuron. We also ensured that it was easy to add new gates to the

[3]An elegent and simple proof of Clingo's Turing completeness falls out of the C-MCPN Framework.

[4]Python 3.8.5 is required (in part?) due to its native support for the tomlib package.

editor. This was an important design choice, as it allows the editor to be easily extended in the future to support a wider range of applications. One more feature of note is the reorganization algorithm. We used a modified version of the Sugiyama algorithm, taking just the layer assignment step and a simplified node positioning step [**Sugiyama1981**].

3.2.1 AI-Assisted Editing

Claude (Anthropic) was used as a programming assistant throughout development of the editor, helping with DearPyGui module integration, debugging, and some code generation. All design decisions, research direction, and domain-specific logic (ASP, gate definitions, network structure) were made by the authors. Every line generated by Claude was reviewed, edited, and approved by the authors before being added to the codebase, and it did not have direct access to the codebase at any point. All of the files Claude had input in are contained in the `app/` directory of the project, though not all files in this directory had input from Claude.

4 Implementation

4.1 C-MCPN Framework

From what we could find in the literature, the C-MCPN Framework is a novel approach to neural network representation within a logical programming paradigm like ASP. The networks themselves have relatively simple representations that can be leveraged to create complex operations. The framework uses three main program types:

- **Gate Definitions Program:** Files are in the `C-MCPN/gates/` directory.
- **Network Structure Program:** Files are in the `C-MCPN/networks/` directory.
- **Value Inference Program:** Files are in the `C-MCPN/base/` directory.

The files in the gate definitions program define the behavior of each gate, including the number of inputs it should have. Every default gate we included also has a special header used to send meta-information to the C-MCPN Editor. For example, the file defining the AND gate looks like this:

```
%% gate: AND
%% inputs: 2
%% outputs: 1
```

```
% type("TYPE")
type("AND").

% gate(TYPE, OUTPUT, INPUTS)
gate("AND", Out, In1, In2) :-
    values(In1), values(In2),
    Out = (In1 * In2).
```

Note the metadata in the header, and the two rules which guide this gate's definition. The first simply defines the gate, and the second performs an operation on its inputs. Then, the files in the network structure program define the neurons which exist and their connections. For example, the file defining the 1-bit adder includes the following lines:

```
% neuron(type, unique_id)
neuron("INPUT", "INPUT_1").
neuron("XOR", "XOR_1").

% edge(source_id, target_id)
edge("INPUT_1", "XOR_1").

% { edge(source_id, target_id) : neuron(type, target_id) } max_inputs.
{ edge(src, "XOR_1") : neuron("XOR", "XOR_1") } 2.

% val(neuron_id, value).
val("INPUT_1", 1).
```

Note the cardinality constraints which enforce the correct number of inputs for each neuron, as well as the fact which sets the value of the input neuron to 1. Finally, the value inference program performs two important roles. It activates each gate by allowing the edges between neurons to carry values, and it assigns each individual neuron a value based on its gate type. The following lines are from the value inference program, and show how this is implemented:

```
%% Value domain
values(0;1).

% Edges carry values from input to output
edge(In, Neuron_id, Val) :-
    edge(In, Neuron_id),
```

```
    neuron(_, Neuron_id),
    neuron(_, In),
    val(In, Val).

% 2 input gates
val(Neuron_id, Val) :-
    gate(Type, Val, Val1, Val2),
    neuron(Type, Neuron_id),
    edge(In1, Neuron_id, Val1),
    edge(In2, Neuron_id, Val2),
    In1 != In2.

%% Each neuron has one value
{ val(Neuron_id, Val) : values(Val) } 1 :- neuron(_, Neuron_id).

%% Show values
#show val/2.
```

Note the first fact which sets the domain of values to be binary and the constraint that ensures each neuron has exactly one value. Most importantly, the rule which carrys the values along the edges is vital to the functioning of the network. Without it, the network gets "stuck" and is unable to infer what the values of the neurons should be. This is a key insight we had during development.

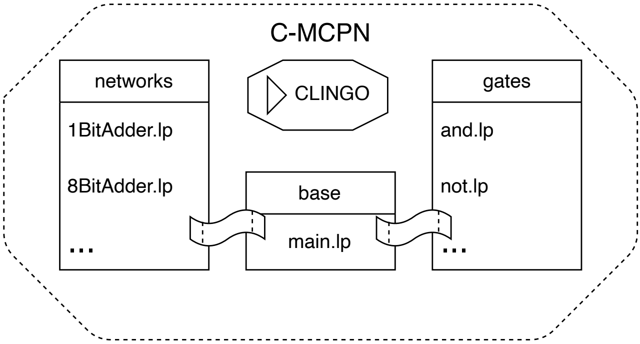


Figure 1: A visualization of the programs involved in the C-MCPN Framework.

4.2 C-MCPN Editor